

STRATEGY REPORT

Ingestion strategy: CSV and PDF

One approach for two kinds of source data: validate first, keep source truth, write atomically, and prove it. Every decision, the reasons for scripts over CRUD, and the questions we left open on purpose.

IMPLEMENTED / VERIFIED / DEMO SCOPE

PROJECT

AI-assisted SupportOps

REPORT DATE

October 2, 2026

SCOPE

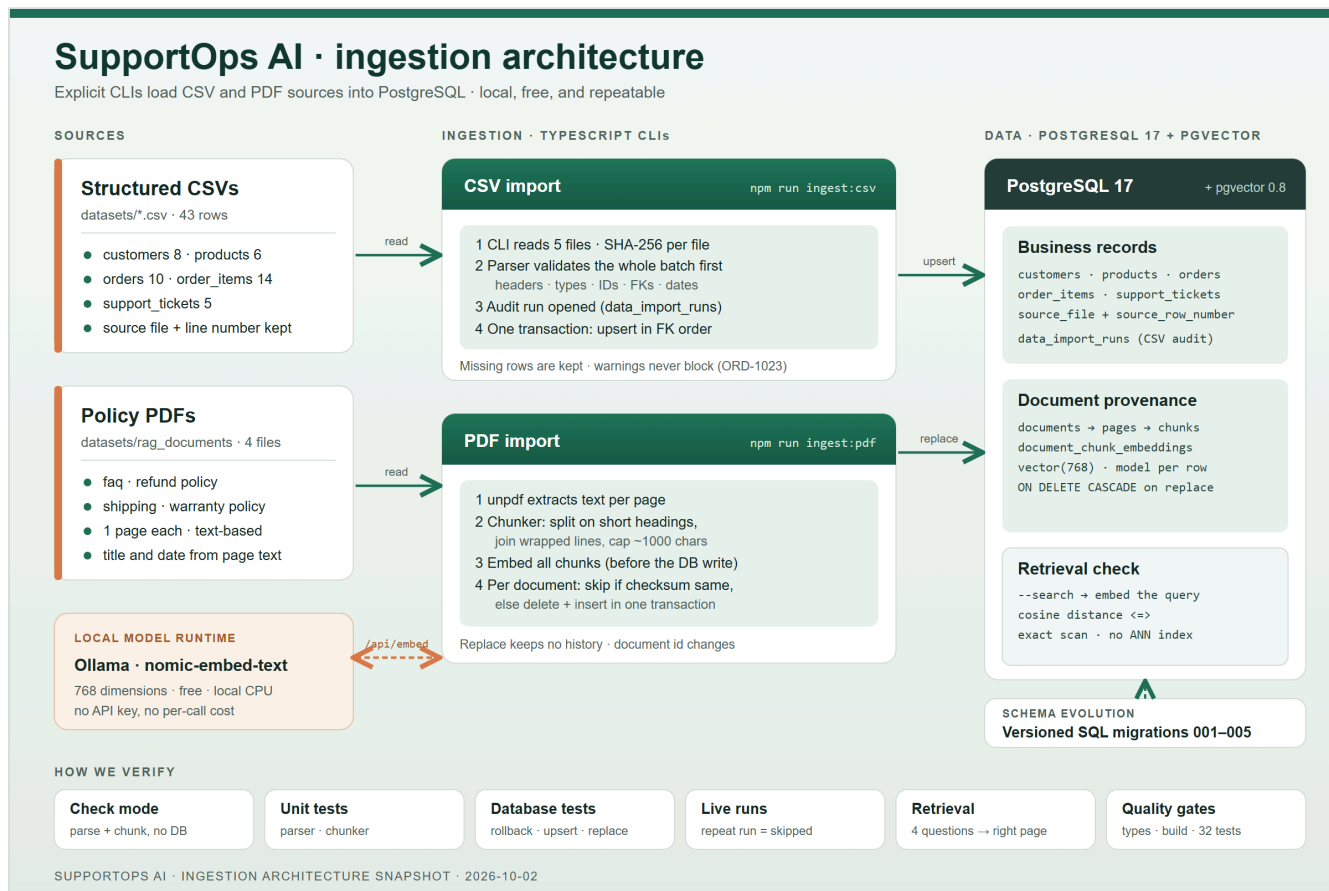
Structured CSV and document PDF ingestion

Learn how the pieces fit, at zero cost

This project is a learning exercise. The aim was to understand how data gets from supplied files into a database in a form an AI-assisted support tool can later use, not to build a production data platform.

- **Structured data (five CSVs, 43 rows)** becomes relational tables with keys, constraints and provenance.
- **Documents (four PDFs)** become pages, chunks and vector embeddings for later retrieval.
- **Constraint:** everything must run free and locally, which drove the choice of a local embedding model.

The whole ingestion process



Two sources, two import scripts, one database. CSV rows are validated as a batch and upserted in one transaction. PDFs are chunked, embedded by a local Ollama model, and written per document. The bottom row shows how each path is verified.

Orange marks sources and the local model. Green marks application code. The dark card is PostgreSQL 17 with the pgvector extension.

Same principles, different mechanics

ASPECT	CSV (STRUCTURED)	PDF (DOCUMENTS)
Command	<code>npm run ingest:csv [-- --check]</code>	<code>npm run ingest:pdf [-- --check --search "..."]</code>
Code layout	CLI, import service, pure batch parser	CLI, import service, extractor, chunker, embedding client
Parsing	<code>csv-parse</code> ; exact headers, types, enums, dates	<code>unpdf</code> text per page; heading-based chunker
Validation	Whole batch: duplicates, foreign keys, ticket-to-order ownership, date order	Non-empty text; embedding count and 768-dimension check
Write unit	One transaction for all five tables	One transaction per document
Repeat run	Upsert by source id; omitted rows kept	Skip if checksum unchanged; replace if changed
Provenance	Source file path and physical CSV line on every row	Path, SHA-256, page number, heading
Audit	Persistent <code>data_import_runs</code> table	None yet (CLI output only)
Result	8 customers, 6 products, 10 orders, 14 items, 5 tickets	4 documents, 4 pages, 12 chunks, 12 embeddings

Principle shared by both: keep the original source independently traceable, and treat everything derived (chunks, vectors) as regenerable from it.

Every decision, with its reason

ID	DECISION	REASON AND CONSEQUENCE
A1	Scripts run by an operator, not API endpoints or startup seeding	Explicit, reproducible and testable. No authentication, upload UI or queue to build. Long batch work stays out of request handlers.
A2	CLI, then a reusable import service, then pure functions	The service owns transactions and can later be wrapped by an endpoint or a scheduled job. Pure parsing and chunking test without a database.
A3	A check-only mode on both CLIs	Validate or chunk without a database or model, which is quick feedback and safe to run anywhere.
A4	Source truth before derived data	Original files stay traceable. Chunks and embeddings can be rebuilt. Embeddings live in their own table, keyed by chunk and model.
C1	Validate the whole CSV batch before any business write	Errors from all five files are reported together with file, line and field. Database constraints are a second line of defense.
C2	One transaction for all five tables, atomic or nothing	No half-loaded dataset. A failure rolls back business rows while a separate audit row records the failure with a sanitized SQLSTATE.
C3	Upsert by source id; do not delete omitted rows	Repeatable loads that update in place. A deletion in a later file never propagates (see open questions).
C4	Persistent run audit (migration 004)	Each write attempt records status, timestamps, file checksums, counts, warnings and failure summary. Interrupted runs may stay marked running.
C5	Physical-line provenance; source values preserved; money as decimal strings	Any row traces to its exact CSV line, even with quoted multiline fields. No floating-point conversion. ORD-1023 (header 278.99, items 228.99) is warned about, not "fixed".
P1	Extract with unpdf (pinned 1.7.0); chunk by heading	Fits short, well-structured policies and keeps each chunk citable. The pin avoids raising the Node floor to 22. The heuristic would not generalize to arbitrary PDFs.
P2	Local Ollama nomic-embed-text (768 dimensions)	Free, private, CPU-friendly. Dimension is fixed in the column, so a model change means re-embedding.
P3	pgvector with exact cosine search, no ANN index	Twelve chunks scan instantly with perfect recall. Add HNSW only when the data grows.
P4	Embed before the transaction; one transaction per document	No network call inside a transaction. A failing file does not undo others.
P5	Replace a changed PDF; skip an unchanged one (SHA-256)	Simple and safe to rerun, at the cost of version history.
I1	pgvector/pgvector:pg17 image plus Ollama in Docker Compose	The official Postgres image lacks pgvector. Same major version keeps the existing data volume.

How this would look in a real product, and why we did not build it that way

In a production system, data rarely arrives by someone running a script. It flows through application features. We deliberately did not build those, because the purpose was to learn how ingestion fits together, not to build the surrounding product.

CONCERN	IN A REAL PRODUCT	IN THIS DEMO
Structured records	Customers, products, orders and tickets are created and changed through CRUD endpoints with authorization, validation, business rules, transactions and change history. Deletes are usually soft.	The CSVs are a one-time seed dataset, not a live system of record, so a script loads them.
Bulk CSV import	An upload endpoint stores the file and queues a background job. Rows load into a staging table, are validated, previewed with a downloadable error report, then committed. Progress is visible and the job is resumable. Admin only.	An operator runs a command and reads the printed report.
Document ingestion	A multipart upload with size and type limits and a malware scan, objects in storage, an async worker for extraction and embedding, a status field, re-index and delete actions, and versioning.	Four files in a folder and one command.
Access control	Role-based permissions on who may import or delete.	Whoever can run the script on the machine.
Scale and parsing	Streaming parsers, batching, rate limits, retries, locale and encoding handling.	Tiny files parsed in memory.
Operations	Scheduling, monitoring and alerting.	Manual runs with tests and audit rows.

Why scripts were the right choice now

- **Focus:** CRUD would add authentication flows, forms, queues and storage that are unrelated to learning ingestion.
- **Data shape:** the datasets are static assignment files, with no ongoing writes to model.
- **Reproducibility and testing:** a script is explicit, repeatable and easy to cover with tests.
- **Cost:** no extra services, no hosting, no paid components.

What carries over to a real product: whole-batch validation, atomic transactions, provenance, an audit trail, idempotent re-runs, and a service layer. The import services are written so an endpoint or a job could call them later.

What we left unresolved, on purpose

Replacement strategy

CSV: upsert, full replace, or reject duplicates?	
CHOSEN	Upsert by source id; rows missing from a later file stay.
OPEN	A record deleted in the source is never deleted here. A full-snapshot sync or a tombstone column would be needed to propagate deletions. Current prices and stock are overwritten, while order lines keep their own unit-price snapshot.

PDF: replace or version?	
CHOSEN	Delete and recreate a changed document.
OPEN	The document id changes and history is lost, so old citations go stale. A real product needs a stable document key with versions and effective dates, and chunk-level change detection to avoid re-embedding everything.

All-or-nothing, or per item?	
CHOSEN	CSV is atomic across all files. PDFs are atomic per document.
OPEN	The difference is deliberate but untested against real workloads, and a failed PDF batch can be partly applied.

CSV parsing

How strict and how scalable should parsing be?	
CHOSEN	Exact headers, UTF-8 only, comma delimiter, whole file parsed in memory, aggregated errors with physical line numbers.
OPEN	Large files need streaming. Real uploads need column mapping, delimiter and encoding detection, locale-aware numbers and dates, and partial-accept policies. Skipping unchanged files by checksum, as the PDF path does, is not done for CSVs.

Other

Data meaning and housekeeping	
OPEN	Currency is unspecified in the data. Customer tiers and product categories are closed lists, so a new value is rejected and needs a migration. Stale <i>running</i> audit rows have no reconciliation. The PDF path has no audit table, no concurrency lock, and no handling for files removed from the folder.

Options and why we used a free local model

OPTION	WHERE	NOTES
<code>nomic-embed-text</code> (768)	Local	Chosen. About 274 MB, free, CPU-friendly, with task prefixes for documents versus queries.
<code>all-minilm</code> (384)	Local	Smaller and faster, lower quality.
<code>mxbai-embed-large</code> (1024)	Local	Heavier, typically better quality.
<code>bge-m3</code> (1024)	Local	Multilingual.
OpenAI <code>text-embedding-3-small</code> / <code>-large</code>	Hosted	Strong and simple to call; 1536 and 3072 dimensions. Costs money, needs a key, and sends text out.
Cohere, Voyage, Google	Hosted	Similar trade-offs; models can be deprecated.

Check current provider documentation for model names, dimensions and prices before relying on this table.

- **Why free and local:** the demo must cost nothing, nothing should leave the machine, and a local model makes every step visible.
- **Why not a bigger model:** four short documents do not need one. Sensible chunking matters more at this scale.
- **Constraints to remember:** the vector column is fixed at 768 dimensions; pgvector indexes support up to 2000 dimensions; queries and documents must use the same model.
- **Chat model:** a separate and later choice. It generates answers, produces no stored vectors, and has no schema impact.

Practical costs we hit: the Ollama Docker image is about 9 GB on disk, and the first pull was slow and failed once on a network handshake. A system with limited disk or bandwidth might prefer installing Ollama directly.

Evidence at each layer

AREA	CHECK	RESULT
CSV parser	Unit tests: headers, quoting, multiline physical line numbers, invalid date and decimal, duplicates, foreign keys, ticket ownership, date order, total-mismatch warning	Pass
CSV persistence	Database tests: forced constraint failure rolls back the earlier insert, and the failed audit row survives with only the SQLSTATE; re-import updates matching ids and keeps omitted rows	Pass
CSV live runs	-- check on the real files, then real imports	43 rows validate; three audited successful runs; counts 8 / 6 / 10 / 14 / 5; ORD-1023 warned
PDF chunking	-- check and chunker unit tests on the real PDFs	12 chunks, three per file; a test exposed and fixed a heading bug
PDF persistence	Database tests with a fake embedder: import, skip, replace with no orphans, rollback, nearest-first search	Pass
Embeddings	One real request to local Ollama; live import run twice	768 dimensions; 4 imported, then 4 skipped
Retrieval	Four questions through the search command	All four returned the expected section and page; distances 0.176 to 0.385
Whole project	Strict typecheck, production build, full test suite	Pass; 32 tests in 3 files

Limits of this verification: four sample questions are a smoke test, not a retrieval evaluation. The first question ("How long do refunds take?") returned the 30-day refund window; the PDFs do not state a processing time, so the match is relevant but not a full answer. All tests and runs used a temporary Node 22 runtime, because the installed Node is 18.16.0 and the test toolchain needs a newer one.

Next steps

- Add a PDF run-audit table and an advisory lock.
- Decide on versioning for replaced documents.
- Add keyword search or an ANN index only if retrieval or data size demands it.
- Connect retrieval to the chat endpoint, then choose a chat model.